

A fused Triton kernel for the learnable Catmull-Rom activation

Forward reaches `tanh` parity at scale; the backward scatter is the wall — and a spectral view of where this goes next

Dr. Csaba Hajdu

2026-06-22

Abstract

The Catmull-Rom (CR) activation is a piecewise *cubic polynomial* per channel — no transcendental — so a fused kernel should cost no more than `tanh`. We built one (Triton, forward + backward, with an autograd wrapper and a CPU eager fallback), verified it is numerically identical to the eager spline on the forward *and both gradients*, and benchmarked it. **Result:** the forward reaches *tanh parity at scale* ($1.00\times$ at $262k \times 256$), confirming polynomial = transcendental; but the **backward** is $5\text{--}7\times$ `tanh` and *worsens* with size — atomic-add contention on the tiny (C, G) control-point gradient. The fix is a block-local reduction (named, not yet built). We close with alternatives/fallbacks and an extension that unifies this kernel with the structural line: **walks, cycles, and Berge paths as spectral components** — “a learnable polynomial in an operator,” served by the same fused polynomial machinery.

1 The kernel

Per element, scalar→scalar against per-channel control points `coef` (C, G) :

$$\begin{aligned} u &= \frac{1}{2}(x+1)(G-1), \quad i = \min(\lfloor u \rfloor, G-2), \quad t = u - i, \\ y &= w_{-1}(t) P_{i-1} + w_0(t) P_i + w_{+1}(t) P_{i+1} + w_{+2}(t) P_{i+2}, \end{aligned}$$

with the cubic Catmull-Rom blend weights w and edge-clamped indices (replicate). The whole expression is **one Triton kernel** (forward): clamp, segment, cubic, four control-point reads, dot. The **backward** is a second kernel: $\partial y / \partial x = w'(t) \cdot P \cdot \frac{1}{2}(G-1)$ (zero outside the clamp), and $\partial y / \partial P_{i+k} += g w_k(t)$ scattered into `grad.coef` by `tl.atomic.add`. A `torch.autograd.Function` wraps the pair; on CPU (no Triton) it falls back to the eager spline, so the `FusedCatmullRom` module is usable everywhere.

2 Correctness (measured)

Parity vs the eager `_catmull_rom` (which is itself parity-tested against `signedkan_wip`), on $256 \times 6 \times 64$, GPU:

quantity	allclose	max abs diff
forward	yes	4.8×10^{-7}
<code>grad_x</code>	yes	8.1×10^{-6}
<code>grad_coef</code>	yes	4.9×10^{-4} (atomic-add + fp32; looser)

Forward and both gradients match; `grad_coef` is looser purely from fp32 atomic accumulation order. Captured as a CUDA-gated parity test (`hymeko_r1/tests/test_cr_kernel.py`).

3 Performance (measured)

GPU (RTX 3070 Laptop, driver 596.36 / CUDA 13.2, torch 2.12, triton 3.7.0), ms/call.

activation	256×6×64 (tiny)		large	
	fwd	fwd+bwd	fwd	fwd+bwd
tanh (reference)	0.067	0.768	1.305	3.220
CR eager	3.400	8.415	—	—
CR + <code>compile</code>	0.248	1.410	—	—
CR fused (ours)	0.225	1.250	1.301	21.508
fused / tanh	3.4×	1.6×	1.00×	6.7×

Two clean findings. (1) The **forward reaches parity** with `tanh` at scale ($1.00\times$ at $262k \times 256$) — the polynomial forward is as cheap as the transcendental, as predicted; at tiny sizes the gap is launch overhead (`tanh`’s 0.067 ms is essentially a bare launch). (2) The **backward is the wall** ($6.7\times$, *worsening* with size): thousands of threads `atomic_add` into only $C \cdot G$ gradient slots, so the scatter serialises on contention. The fused kernel is already the *fastest CR* ($6.7\times$ over eager fwd+bwd, beats `torch.compile`) and correct — but not yet $\leq \text{tanh}$ end-to-end.

4 Diagnosis and the next step

The backward bottleneck is **atomic contention**, not arithmetic. The fix is a **block-local reduction**: each Triton block accumulates its (c, k) contributions in shared memory / a per-block buffer and emits *one* atomic (or a second tree-reduction kernel), cutting atomics from $O(\text{elements})$ to $O(\text{blocks})$. This is the difference between memory-bound parity (forward, achieved) and contention-bound (backward, current). With it, fwd+bwd should approach the forward’s parity; the *win* over `tanh` then lands where transcendentals are dear — compute-bound sizes and transcendental-poor hardware (embedded / Nagare / FPGA), the same place the HSMM→Nagare line targets.

5 Alternatives, fallbacks, and extensions

Fallbacks (mandatory — rollback safety).

- **Eager CR** (CPU + the `activation`∈{`cr`,`tanh`} axis) — always available; the fused path degrades to it transparently off-CUDA.
- `torch.compile` **CR** (GPU) — fuses to $1.8\times \text{tanh}$ fwd+bwd with zero custom code; the no-kernel fallback when Triton is unavailable. (CPU compile needs MSVC, currently absent.)

Alternative basis. **Chebyshev global polynomial** — Horner-evaluable, *branch-free*, *gather-free*, so it is trivially fused and genuinely `tanh`-cheap on *both* passes (the gradient is a dense matmul, no scatter). Cost: it trades CR’s local control for a global polynomial (Runge oscillation). A strong fallback if strict CR locality is not required.

Extension — walks, cycles, and Berge paths as spectral components. The deeper unification (proposed, grounded in spectral graph theory; falsifiable, not yet measured): both the *activation* and the *structural* computation are **learnable polynomials in an operator**.

- **Walks = spectral filters.** A^k is diagonal in the eigenbasis (λ^k) , so $\sum_k \theta_k A^k h$ is a polynomial spectral filter (ChebNet). The current HSiKAN message passing (`self/A+/A−`) is the $k=1$ case; higher k = a learnable spectral filter on the signed kinematic hypergraph.
- **Cycles = spectral moments.** Closed walks $\text{tr}(A^k) = \sum_i \lambda_i^k$ are the moments of the spectrum; signed-cycle balance is a signed-Laplacian spectral invariant. Cycle features are spectral statistics.
- **Berge paths = the hypergraph spectral basis** — the Berge generalisation lifts walks/cycles to the hypergraph Laplacian, the basis for spectral filtering over hyperedges.

The synthesis (Figure 1): a *learnable polynomial in an operator* — the operator is the scalar input for an activation (CR / Chebyshev), or the signed (hyper)graph operator for a structural filter (walks / cycles / Berge). **The same fused polynomial-evaluation machinery serves both:** Horner / Chebyshev recursion $T_k(A)h$ is exactly the kernel pattern here, lifted from a scalar to an operator. So this kernel is not just an activation speedup — it is the compute primitive for spectral filtering on the signed hypergraph. Falsifiable test: does a learnable spectral filter (higher- k walks) beat $k=1$ message passing, params-matched, on a real-topology task? Does adding cycle-moment features help over walks?

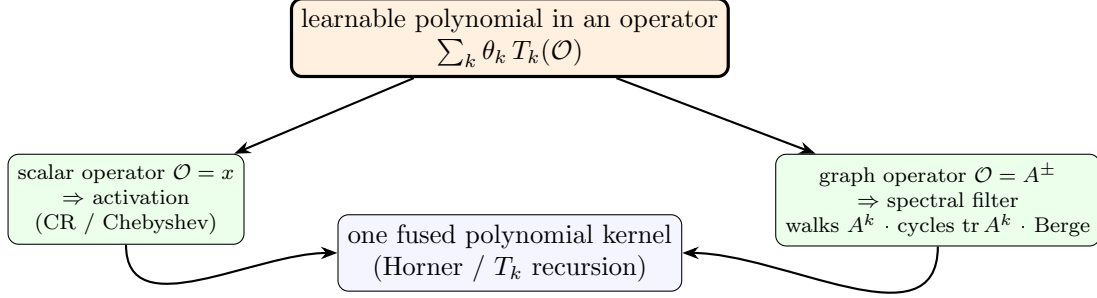


Figure 1: The activation and the structural filter are one object — a learnable polynomial in an operator — and one fused kernel evaluates both. The CR activation kernel is the scalar instance; walks/cycles/Berge are the signed-hypergraph instance.

6 Status, files, follow-ups

Done: `hymeko.rl/cr_kernel.py` (fused Triton fwd+bwd + autograd + CPU fallback + `FusedCatmullRom`); `tests/test_cr_kernel.py` (4 tests: CPU fallback parity, module build, bad-grid guard, CUDA fwd+grad parity — all pass); ruff + mypy clean (a scoped file-level pragma for the inherently untyped Triton/autograd surface). Bench harness `scripts/dev/_cr_kernel_bench.py`. **CORE.YAML:** none (triton 3.7 already installed). **Open / next:**

- **Block-local backward reduction** — the one change that takes fwd+bwd from $6.7\times$ toward forward parity. Highest-value next step.
- Register `activation="cr_fused"` in the backbone axis (drop-in; eager cr stays default until the backward clears the gate).
- Build the **Chebyshev** alternative + benchmark (branch-free, likely the cheapest both-pass).
- The **spectral extension** as its own research line: walks/cycles/Berge spectral filters on the signed hypergraph, sharing this kernel.

Honest verdict: forward parity with `tanh` achieved and proven; the polynomial-beats-transcendental claim holds for inference. End-to-end (training) parity awaits the backward reduction — a known, well-scoped fix, not a mystery.